

React Js Assignment

Lists and Keys

Question 1: How do you render a list of items in React? Why is it important to use keys when rendering lists?

=>

Rendering a list in React: Use the .map() method to iterate over an array and return a JSX element for each item.

Importance of keys: Keys help React identify which items have changed, been added, or removed, improving performance and ensuring correct rendering.

Question 2: What are keys in React, and what happens if you do not provide a unique key?

=>

Keys in React: Keys are unique identifiers for elements in a list. Without unique keys, React may not update the DOM correctly, leading to bugs or inefficient rendering.

Forms in React

Question 1: How do you handle forms in React? Explain the concept of controlled components.

=>

Handling forms in React: Use state to track form input values and update the state with event handlers like onChange.

Controlled components: Form elements whose values are controlled by React state are called controlled components. The value of the input is set by state, and updates are handled by React, ensuring a single source of truth.

Question 2: What is the difference between controlled and uncontrolled components in React?

=>

Controlled Components: The component's value is controlled by React state. The value attribute is set by state, and changes are handled via onChange. Example: `<input value={state} onChange={handleChange} />`.

Uncontrolled Components: The component's value is managed by the DOM itself. You use refs to access the value directly. Example: `<input ref={inputRef} />`.

Key difference: Controlled components provide a single source of truth (state), while uncontrolled components rely on the DOM for value management.

Lifecycle Methods (Class Components)

Question 1: What are lifecycle methods in React class components? Describe the phases of a component's lifecycle.

=>

Lifecycle methods in React class components: These are special methods that allow you to execute code at specific points in a component's lifecycle.

Phases of a component's lifecycle:

- **Mounting:** When the component is created and added to the DOM. Methods: `constructor()`, `componentDidMount()`.
- **Updating:** When the component's state or props change. Methods: `componentDidUpdate()`.
- **Unmounting:** When the component is removed from the DOM. Method: `componentWillUnmount()`.

Question 2: Explain the purpose of `componentDidMount()`, `componentDidUpdate()`, and `componentWillUnmount()`.

=>

Purpose of specific lifecycle methods:

- **`componentDidMount()`:** Called after the component is added to the DOM. Used for side effects like fetching data or setting up subscriptions.
- **`componentDidUpdate()`:** Called after updates to props or state. Used to perform actions in response to changes.

- **componentWillUnmount():** Called before the component is removed from the DOM. Used for cleanup, such as removing event listeners or canceling network requests.

Hooks (useState, useEffect, useReducer, useMemo, useRef, useCallback)

Question 1: What are React hooks? How do useState() and useEffect() hooks work in functional components?

=>

React Hooks: Hooks are functions that let you use state and other React features in functional components without needing class components.

useState():

- Used to add state to a functional component.
- Returns an array with the current state and a function to update it.

Example:

```
const [count, setCount] = useState(0);  
setCount(count + 1);
```

useEffect():

- Used to handle side effects (e.g., fetching data, subscriptions).
- Runs after the component renders or when dependencies change.
- The dependency array controls when the effect runs. If empty, it runs only once after the initial render.

Example:

```
useEffect(() => {  
  console.log("Effect ran");  
  return () => console.log("Cleanup on unmount or dependency change");  
}, [dependency]);
```

Question 2: What problems did hooks solve in React development? Why are hooks considered an important addition to React?

=>

Problems solved by hooks:

- **State in functional components:** Previously, only class components could manage state, but hooks introduced state management to functional components.
- **Code reuse:** Hooks like `useCustomHook` simplify sharing logic without the need for complex patterns like higher-order components (HOCs) or render props.
- **Lifecycle complexity:** Managing lifecycle methods in class components often led to unrelated logic being combined or duplicated. Hooks like `useEffect` group related logic together.

Why hooks are important:

- Enable functional components to be more powerful and concise.
- Simplify code, making it easier to read, test, and maintain.
- Provide a modern, flexible way to manage state and side effects.
- Encourage consistent patterns, improving the overall developer experience.

Question 3: What is `useReducer`? How we use in react app?

=>

`useReducer` is a React hook used for managing complex state logic in functional components.

It is an alternative to `useState` and is especially useful when state transitions depend on actions or when state logic involves multiple sub-values.

How `useReducer` works:

- It takes two arguments: a reducer function and an initial state.
- Returns the current state and a dispatch function to trigger state updates.

Question 4: What is the purpose of `useCallback` & `useMemo` Hooks?

=>

useCallback: Optimizes performance by memoizing a function, preventing it from being recreated on every render unless its dependencies change. Useful for passing stable functions to child components.

useMemo: Optimizes performance by memoizing a computed value, recalculating it only when its dependencies change. Useful for expensive calculations to avoid unnecessary recomputation.

Question 5: What's the Difference between the useCallback & useMemo Hooks?

=>

The key difference between **useCallback** and **useMemo** lies in what they optimize:

- **useCallback:** Returns a memoized **function** and is used to prevent unnecessary re-creations of functions.
- **useMemo:** Returns a memoized **value** and is used to avoid unnecessary recalculations of expensive computations.

In short, **useCallback** is for functions, and **useMemo** is for values.

Question 6 : What is useRef ? How to work in react app?

=>

useRef is a React hook that provides a way to persist values across renders without triggering re-renders. It is commonly used to access DOM elements or store mutable values that don't require a re-render when updated.

- **useRef** is persistent across renders but doesn't trigger re-renders when updated.
- **useRef** returns an object with a **.current** property that you can modify.